Reinforcement Learning for Understanding DeepSeekMath

From Bandits and Bellman Equations to PPO, DPO, and GRPO

Intuitive study note

April 2026

Intuition

Reinforcement learning is the study of how an agent can learn by *trying actions*, *seeing outcomes*, and then *shifting probability mass toward decisions that pay off more often*. For LLM reasoning, the actions are tokens, the "environment" is the prompt plus generated prefix, and the reward often arrives only at the end.

All figures in this note are original schematic redrawings rather than copied images. They are inspired by canonical expositions such as Sutton and Barto's textbook, the DQN paper, PPO/TRPO material, OpenAI Spinning Up, and the DeepSeekMath paper.

Contents

1 Why RL Feels Different **2**

2 Start Even Smaller: Bandits **2**

3 From Bandits to Full RL: Markov Decision Processes **3**
- 3.1 The Smallest RL Picture . . . 3
- 3.2 One Running Example . . . 4

4 How Classical RL Learns **4**
- 4.1 Dynamic Programming . . . 4
- 4.2 Monte Carlo . . . 5
- 4.3 Temporal-Difference Learning . . . 5
- 4.4 SARSA and Q-Learning . . . 5

5 When Neural Networks Enter: Deep RL **6**
- 5.1 DQN and the Value-Based Deep RL Line . . . 7
- 5.2 Policy Gradient: Optimize the Policy Directly . . . 7
- 5.3 Actor–Critic . . . 8
- 5.4 Why PPO Became So Popular . . . 8
- 5.5 Other Major Modern Directions . . . 8

6 A Mental Map of the Major RL Families **9**

7 LLMs as Reinforcement Learning Agents **9**
- 7.1 The Main LLM Post-Training Methods . . . 10

8 Why DeepSeekMath Uses GRPO **10**
8.1 From PPO to GRPO . . . 10
8.2 Outcome Supervision vs. Process Supervision . . . 11
8.3 Why GRPO Is a Natural Fit for Math . . . 12
8.4 What DeepSeekMath Actually Reports . . . 12
8.5 The Most Important DeepSeekMath Insight . . . 13

9 How to Remember the Whole Story **13**

10 If You Want One Sentence per Algorithm Family **14**

11 Further Reading **14**

1 Why RL Feels Different

If supervised learning is "copy the labeled answer," reinforcement learning is "sample a behavior, score the outcome, and then make better-scoring behaviors more likely next time." That single shift explains most of the field.

RL feels hard for three reasons:

- The agent usually sees *delayed rewards*, so it must assign credit backward through a trajectory.
- The agent must *explore*, so it cannot only repeat what already looks best.
- The data distribution depends on the current policy, so learning changes the future data.

Takeaway

No short note can literally enumerate every RL variant that exists today. The useful compression is to understand the main *families*: bandits, dynamic programming, Monte Carlo, temporal-difference methods, value-based deep RL, policy gradients, actor-critic methods, model-based RL, offline RL, and modern LLM post-training methods such as DPO, PPO, and GRPO.

2 Start Even Smaller: Bandits

The multi-armed bandit is RL with the memory removed. You repeatedly choose one of several actions, immediately observe a reward, and want to maximize long-run payoff.

Intuition

Imagine choosing between three study habits each evening: flash cards, worked examples, or rereading notes. There is no changing state, only a repeated choice and a reward the next day. That is a bandit, not a full RL problem.

The central tension is **exploration vs. exploitation**:

- **Exploitation**: choose the arm that currently looks best.
- **Exploration**: try uncertain arms because your estimate might be wrong.

Classic bandit rules already preview later RL ideas:

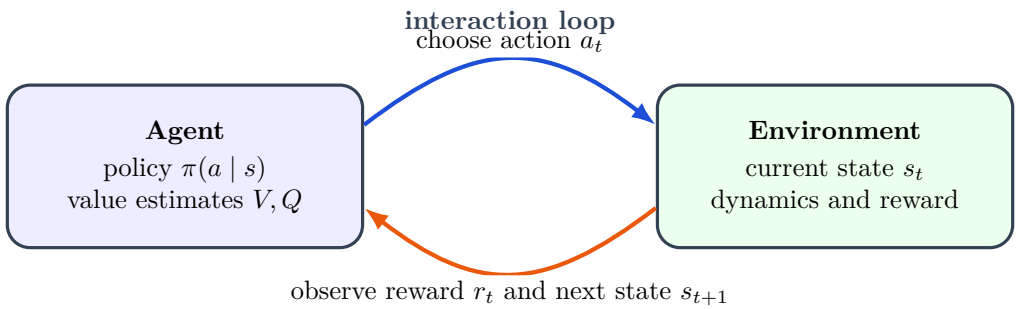

Figure 1: The basic agent–environment loop. This is the common core behind chess engines, robot control, Atari agents, and LLM post-training.

- **ε-greedy**: usually take the current best action, but sometimes explore randomly.
- **UCB**: prefer actions with high estimated reward *and* high uncertainty.
- **Thompson sampling**: sample from your posterior belief and act according to that sampled world.

Math Snapshot

The simplest action-value update is

$$Q_{n+1}(a) = Q_n(a) + \alpha(R_n - Q_n(a)).$$

This says: *move your old estimate toward the new reward.* Nearly all RL updates keep this same skeleton: old estimate plus learning rate times prediction error.

Bandits teach the first durable lesson of RL: *good control requires both reward estimates and uncertainty management.*

3 From Bandits to Full RL: Markov Decision Processes

Bandits have no state. Full RL adds dynamics: your action changes what situation you face next. The standard model is the **Markov Decision Process (MDP)**:

$$(\mathcal{S}, \mathcal{A}, P, R, \gamma).$$

Here $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, P the transition rule, R the reward rule, and $\gamma \in [0, 1)$ the discount factor.

3.1 The Smallest RL Picture

If the current state contains all information needed for prediction, we call the problem *Markov*. Then a policy $\pi(a \mid s)$ fully specifies behavior.

The return from time t is

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots .$$

Why discount? Three intuitive reasons:

- It makes "reward now" worth more than "reward much later."
- It helps infinite-horizon sums stay finite.
- It stabilizes planning by reducing how far tiny errors propagate.

Two functions dominate RL theory:

- **State value** $V^\pi(s)$: expected return if you start in state s and then follow policy π.
- **Action value** $Q^\pi(s,a)$: expected return if you take action a in state s and then follow π.

Math Snapshot

The Bellman expectation equations are the self-consistency laws of RL:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} P(s', r \mid s, a)\,[r + \gamma V^\pi(s')],$$

$$Q^\pi(s,a) = \sum_{s',r} P(s', r \mid s, a)\Big[r + \gamma \sum_{a'} \pi(a' \mid s') Q^\pi(s', a')\Big].$$

The deep idea is that a value function equals *immediate reward plus discounted value of what comes next.*

3.2 One Running Example

Picture a maze. The agent receives -0.01 each step, $+1$ at the exit, and maybe -1 for stepping into lava. Now every algorithm in RL is trying to answer some version of one question:

How should I update my beliefs so that future action choices lead me to the exit faster and more reliably?

4 How Classical RL Learns

The historical flow is simple:

- If you know the model exactly, you can plan with **dynamic programming**.
- If you can run episodes but do not know the model, you can average outcomes with **Monte Carlo**.
- If you want to learn online and bootstrap from current estimates, you use **temporal-difference learning**.

4.1 Dynamic Programming

Dynamic programming assumes the transition model P is known. Then you can repeatedly evaluate and improve a policy.

The two classical loops are:

- **Policy iteration**: evaluate π, then greedify it.
- **Value iteration**: repeatedly apply the Bellman optimality backup until values stabilize.

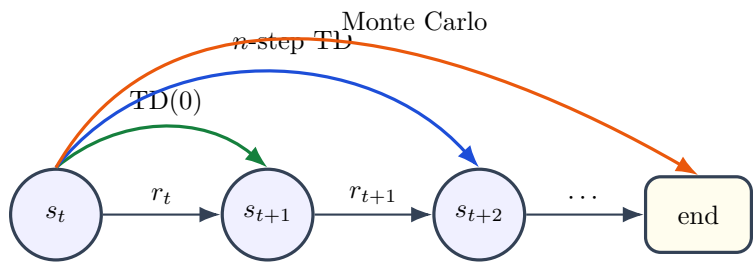

Figure 2: How backup depth changes the algorithm. TD updates after one step, Monte Carlo waits until termination, and n-step methods interpolate between them.

Intuition

Dynamic programming is like having the full rules of a board game and computing consequences before acting. It is powerful, but in many real problems you do not know the transition table or the state space is too large.

4.2 Monte Carlo

Monte Carlo (MC) waits until an episode ends, then uses the observed return to update values. It does not bootstrap from current value estimates.

Pros: unbiased with enough samples and conceptually simple.
Cons: high variance and awkward for long episodes because you must wait until the end.

4.3 Temporal-Difference Learning

TD learning updates early, before the episode ends. The basic TD(0) update is

$$V(s_t) \leftarrow V(s_t) + \alpha\Big(r_t + \gamma V(s_{t+1}) - V(s_t)\Big).$$

The quantity in parentheses is the **TD error**. It is the same basic form as the bandit prediction error, except now the target includes a bootstrap term from the next state's current value estimate.

Takeaway

Monte Carlo learns from complete outcomes. TD learns from partial outcomes plus current guesses. That single difference creates the bias–variance trade-off at the heart of classical RL.

4.4 SARSA and Q-Learning

Once we move from prediction to control, the central object becomes $Q(s, a)$.

SARSA is on-policy:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\big].$$

Q-learning is off-policy:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big].$$

Method	What it assumes	Main strength	Main weakness
Dynamic programming	Known transition model	Exact planning on small MDPs	Unrealistic for most large unknown environments
Monte Carlo	Episodic rollouts	Simple and unbiased in the limit	High variance, must wait for episode end
TD(0), TD(λ)	Streaming experience	Data efficient, online updates	Bootstrapping can propagate bad estimates
SARSA	On-policy control	Safer under exploratory behavior	Can be conservative
Q-learning	Off-policy control	Strong asymptotic target, greedy improvement	Can be unstable with function approximation

Table 1: The classical RL toolbox. These methods are still conceptually central even when modern implementations use neural networks.

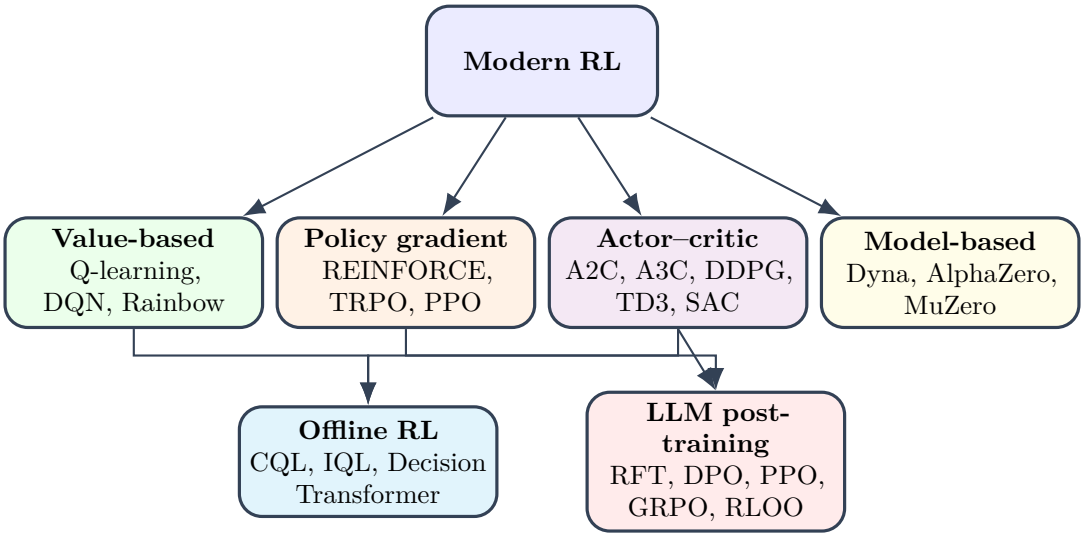

Figure 3: A family tree of the major modern RL directions. This is a conceptual map, not an exhaustive list of every named variant.

The intuitive difference is clean:

- **SARSA** learns about the action you actually took next. It is usually more conservative.
- **Q-learning** learns about the best next action according to your current estimate. It is more optimistic and often more aggressive.

5 When Neural Networks Enter: Deep RL

Tabular methods break when the state space is huge. You cannot store one value per pixel image, robot pose history, or token prefix. The fix is function approximation: use a neural network to approximate values or policies.

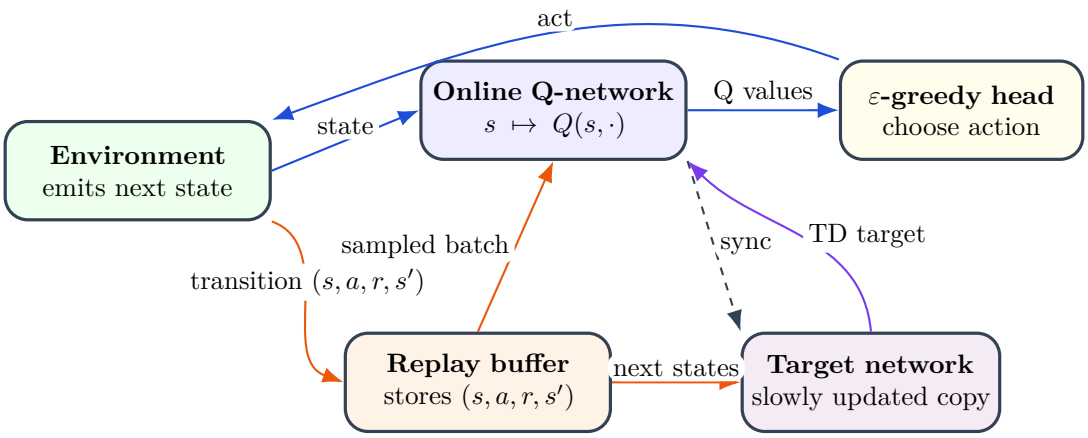

Figure 4: Original schematic of the DQN training loop, inspired by the stabilizing ideas in the Atari DQN line.

5.1 DQN and the Value-Based Deep RL Line

Deep Q-Networks (DQN) combine Q-learning with a neural network. The network receives a state and outputs values for actions. Two stabilizing ideas made DQN practical on Atari [3]:

- **Replay buffer**: store past transitions and train on shuffled minibatches so updates are less correlated.
- **Target network**: use a slowly updated copy of the Q-network to build training targets.

Important descendants include:

- **Double DQN**: reduce overestimation bias.
- **Dueling DQN**: separately represent state value and action advantage.
- **Prioritized replay**: sample more informative transitions more often.
- **Rainbow**: combine the strongest ingredients into one agent.

5.2 Policy Gradient: Optimize the Policy Directly

Value-based methods choose actions by maximizing a learned value. Policy-gradient methods learn a parameterized policy directly.

The canonical score-function result behind REINFORCE is

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t G_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right].$$

The intuition is wonderfully simple:

- If a trajectory turned out well, increase the log-probability of the actions that produced it.
- If a trajectory turned out badly, decrease those probabilities.

Intuition

REINFORCE says: *do more of what lucky trajectories did.* The problem is variance. A single lucky episode can shout too loudly. This is why later methods add baselines, critics, clipping, and trust regions.

5.3 Actor–Critic

Actor–critic methods split the job:

- The **actor** is the policy π_θ.
- The **critic** estimates values and provides a lower-variance training signal.

The key quantity is the **advantage**

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s),$$

which answers: *how much better was this action than my baseline expectation in this state?*

This idea powers A2C, A3C, GAE, DDPG, TD3, and SAC.

- **A2C/A3C**: synchronous or asynchronous actor–critic for faster parallel learning.
- **GAE**: smooth bias–variance trade-off for advantage estimation [4].
- **DDPG / TD3**: deterministic actor–critic methods for continuous control.
- **SAC**: maximize reward *and* entropy, so the policy stays stochastic and exploratory.

5.4 Why PPO Became So Popular

TRPO said: update the policy, but do not move too far in one step. PPO kept that spirit with a simpler clipped objective [6]:

$$L^{\text{PPO}}(\theta) = \mathbb{E}\big[\min(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t)\big],$$

where $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$.

PPO became popular because it is:

- simpler than TRPO,
- more stable than raw policy gradient,
- flexible enough for robotics, games, and later RLHF-style LLM training.

5.5 Other Major Modern Directions

Beyond DQN and PPO, the main modern branches are:

- **Model-based RL**: learn or use a world model, then plan inside it. Dyna pioneered the idea; AlphaZero and MuZero made it famous in high-performance planning systems.

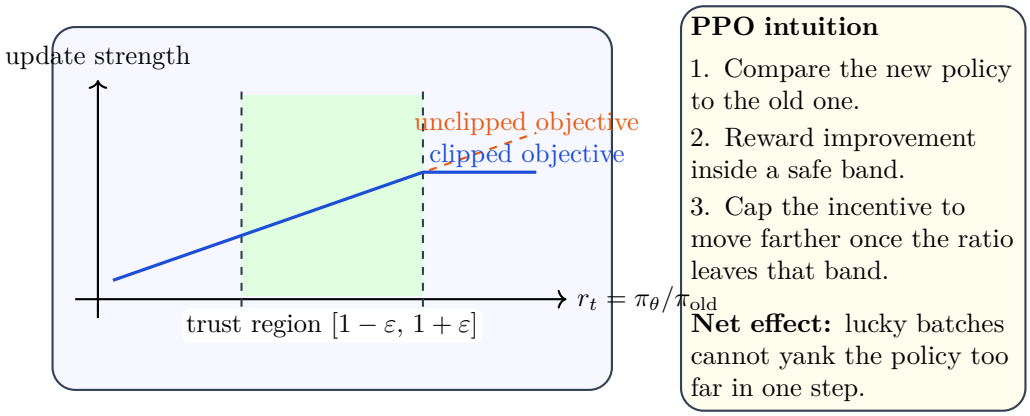

Figure 5: PPO's core intuition: do not let one update overreact to a lucky batch. The clipping acts like a seat belt for policy gradients.

- **Offline RL**: learn from a fixed logged dataset without new interaction. Representative ideas include conservative value learning (CQL), implicit Q-learning (IQL), and sequence-model views such as Decision Transformer.
- **Hierarchical and multi-agent RL**: decompose long-horizon behavior into skills or coordinate multiple agents.

6 A Mental Map of the Major RL Families

7 LLMs as Reinforcement Learning Agents

An LLM can be viewed as an episodic RL policy:

- **State**: the prompt plus the tokens generated so far.
- **Action**: the next token.
- **Episode**: one completion.
- **Reward**: a score for helpfulness, correctness, preference, safety, or final answer quality.

This is both familiar and strange:

- familiar, because it is still a policy over actions;
- strange, because the action space is huge, rewards are sparse, and one bad update can destroy useful language behavior.

That is why LLM RL almost always includes **regularization toward a reference policy**, often the supervised fine-tuned model. In practice, the model is allowed to improve, but not to wander arbitrarily far.

Family	Representative algorithms	Best intuition
Bandits	ε-greedy, UCB, Thompson sampling	Learn rewards under uncertainty when there is no state transition
Planning with known model	Policy iteration, value iteration	Compute consequences before acting
Model-free prediction / control	MC, TD, SARSA, Q-learning	Learn values from trial and error
Value-based deep RL	DQN, Double DQN, Rainbow	Use a neural critic to tell actions how good they are
Policy-based RL	REINFORCE, TRPO, PPO	Push probability mass directly toward better trajectories
Actor–critic	A2C, A3C, DDPG, TD3, SAC	Actor chooses, critic judges
Model-based RL	Dyna, AlphaZero, MuZero	Spend computation on imagination and planning
Offline RL	CQL, IQL, Decision Transformer	Learn from logs while avoiding unsupported actions
LLM RL / alignment	RFT, DPO, PPO, GRPO, RLOO	Reweight sampled completions using preference or reward signals

Table 2: The shortest honest overview of "all the RL algorithms we have now" is really a map of families and their representative members.

7.1 The Main LLM Post-Training Methods

DeepSeekMath gives a useful unifying lens: every post-training method can be understood by three ingredients.

1. Where does the data come from?
2. What acts as the reward signal?
3. How do we convert that signal into a gradient coefficient?

What about newer names such as IPO, ORPO, SimPO, KTO, RLOO, ReMax, or RAFT? They mostly modify one of the same axes: the data source, the reward/preference signal, or the exact form of the gradient coefficient. The family structure matters more than memorizing every acronym.

8 Why DeepSeekMath Uses GRPO

DeepSeekMath is about mathematical reasoning, so the RL question becomes:

Given several candidate solutions to the same math problem, how do we reinforce the reasoning patterns that look better without paying the full memory cost of PPO?

Their answer is **Group Relative Policy Optimization (GRPO)** [16].

8.1 From PPO to GRPO

In LLM PPO, you typically need:

- a **policy model** to generate text,
- a **reward model** to score text,
- a **value model / critic** to estimate advantages,

Method	Data source	Reward signal	What it updates	Intuition
SFT	Human demonstrations	implicit human choice	policy	imitate good behavior
RFT	Sampled completions, keep the correct ones	rule-based correctness or filtering	policy	train only on accepted outputs
Online RFT	Completions from the current policy	rule-based correctness	policy	keep refreshing the accepted set online
DPO	Preference pairs	pairwise winner vs. loser	policy	make preferred completions more likely without an explicit reward model
PPO	Online samples from current policy	reward model + critic-based advantage	policy and critic	RLHF with trust-region style updates
GRPO	Online groups of samples for the same prompt	reward model with group-relative normalization	policy only	compare completions against each other instead of training a separate critic

Table 3: A useful LLM-alignment map, following the unified perspective emphasized in DeepSeekMath.

- a **reference model** to supply a KL penalty.

That critic is expensive. Worse, LLM rewards are often sparse: a long completion may only receive a useful reward at the end, so token-level value estimation is awkward.

GRPO removes the critic. Instead, for one question q, sample a group of outputs

$$\{o_1, o_2, \ldots, o_G\},$$

score them with a reward model, and normalize *within the group*:

$$\tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}.$$

For outcome supervision, every token in output o_i receives the same advantage signal $\hat{A}_{i,t} = \tilde{r}_i$.

DeepSeekMath Lens

The key intuition is comparative: a reward model is usually better at saying *which of these completions is better for the same question* than at assigning an absolutely calibrated global score. GRPO leans into that fact.

8.2 Outcome Supervision vs. Process Supervision

DeepSeekMath discusses two ways to reward reasoning:

- **Outcome supervision**: only the final answer matters. If the completion ends well, all tokens in that output receive a positive signal.
- **Process supervision**: intermediate reasoning steps are scored, so credit can be assigned more locally along the chain of thought.

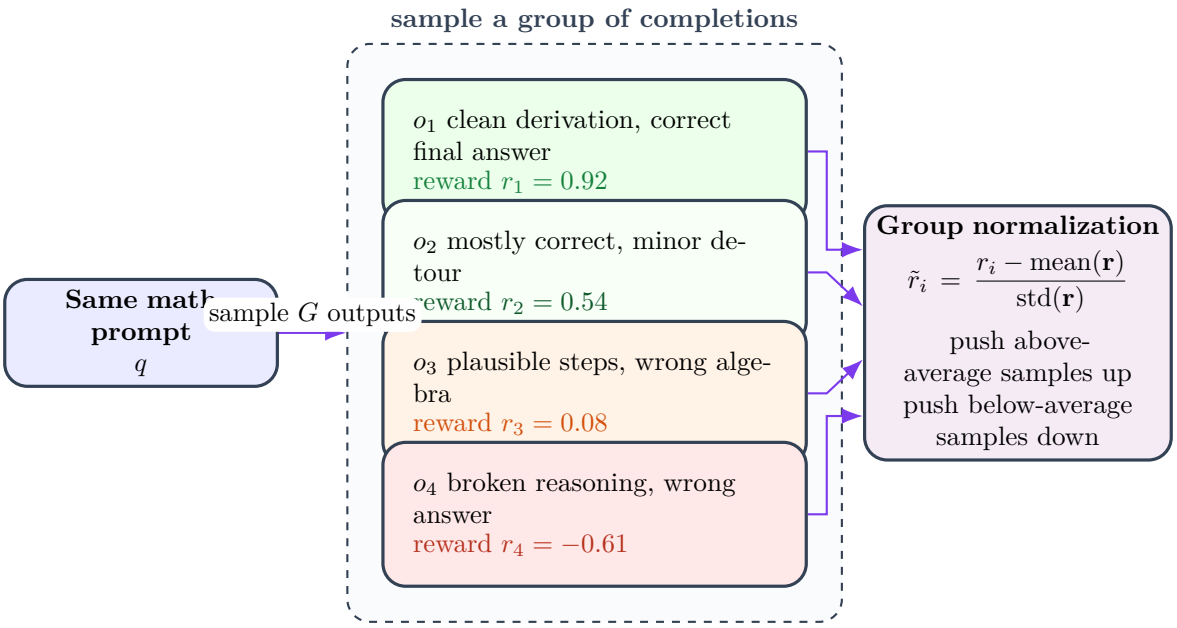

Figure 6: Original intuition figure for GRPO, inspired by the PPO-versus-GRPO comparison in DeepSeekMath. Instead of training a separate critic, GRPO uses relative performance inside a group of completions for the same prompt.

Process supervision is more informative, especially for long mathematical derivations, but it requires a process reward model. Outcome supervision is simpler and still effective.

8.3 Why GRPO Is a Natural Fit for Math

Mathematical reasoning has three traits that make GRPO appealing:

- **Relative comparisons are meaningful**: among 64 sampled solutions to the same problem, some are obviously better than others.
- **Correctness can sometimes be judged by rules**: final answers or verifiers can provide sharp reward signals.
- **Long outputs make token-level critics expensive and noisy**: removing the critic simplifies training.

8.4 What DeepSeekMath Actually Reports

The DeepSeekMath RL stage starts from the instruction-tuned model, uses chain-of-thought math questions from GSM8K and MATH, trains a reward model, and then performs online RL. A few details worth remembering from the paper:[16]

- RL training used roughly 144K chain-of-thought-format questions from their SFT data.
- For each question, they sampled 64 outputs during GRPO.
- They used KL regularization toward a reference model with coefficient 0.04.

Question	PPO answer	GRPO answer
How is advantage computed?	Via a learned critic/value model, often with GAE	Via group-relative rewards among completions for the same prompt
What gets regularized?	Policy stays near reference model, often through KL penalty in the reward/objective	Policy stays near reference model with an explicit KL regularizer in the loss
What is the memory burden?	Higher, because actor and critic both matter	Lower, because no separate critic is trained
What is the main intuition?	Trust-region actor–critic RLHF	PPO-style update with comparative, critic-free baselines

Table 4: The practical difference between PPO and GRPO in the DeepSeekMath setting.

- Their RL model improved not only in-domain benchmarks but also several out-of-domain math benchmarks.

8.5 The Most Important DeepSeekMath Insight

The paper argues that RL improves reasoning mostly by making the model's output distribution *more robust*, not by magically giving it entirely new capabilities. Their evidence is that RL improves **Maj@K** more than **Pass@K**. In plain English:

- the model may already know how to produce a correct solution sometimes,
- RL makes correct reasoning appear more reliably near the top of the model's distribution.

Takeaway

This is the right intuition for DeepSeekMath's RL part: GRPO is not best thought of as "teaching math from scratch." It is better thought of as *reshaping the model's probability distribution so good mathematical trajectories are sampled more consistently.*

9 How to Remember the Whole Story

Here is the compact storyline:

1. **Bandits**: estimate rewards while exploring.
2. **MDPs**: actions change the future state distribution.
3. **Bellman equations**: value is immediate reward plus discounted future value.
4. **MC and TD**: learn values from experience, either after full episodes or by bootstrapping early.
5. **Q-learning and SARSA**: use action values to control behavior.
6. **Deep RL**: replace tables with neural approximators.
7. **Policy gradient / actor–critic**: optimize policies directly with advantage signals.
8. **PPO**: stable policy updates by clipping large changes.
9. **LLM RL**: actions are tokens; rewards are preferences, verifiers, or reward models.

10. **GRPO**: PPO-like online RL without a critic, using group-relative rewards for the same prompt.

10 If You Want One Sentence per Algorithm Family

- **UCB / Thompson**: explore uncertainty intelligently.
- **Policy iteration / value iteration**: plan exactly when the model is known.
- **Monte Carlo**: learn from whole episodes.
- **TD / TD(λ)**: learn now by bootstrapping from current estimates.
- **SARSA**: learn the value of what you actually do.
- **Q-learning**: learn the value of the greedy next move.
- **DQN**: neural Q-learning stabilized by replay and target networks.
- **REINFORCE**: increase the probability of actions in high-return trajectories.
- **Actor–critic**: let a value estimator reduce policy-gradient variance.
- **PPO**: policy gradient with a conservative, clipped update.
- **SAC**: reward-seeking plus entropy-seeking for robust stochastic control.
- **MuZero**: learn a world model useful for planning.
- **Offline RL**: learn from logged behavior while avoiding unsupported actions.
- **DPO**: optimize preferences directly without an explicit reward model at training time.
- **GRPO**: compare groups of completions for the same prompt and reinforce the relatively better ones.

11 Further Reading

If one pass still feels abstract, the best next move is not to memorize more acronyms. It is to anchor each family to a tiny mental movie:

- a slot machine for bandits,
- a maze for MDPs,
- Atari for DQN,
- robot control for actor–critic and SAC,
- text generation with preference scores for PPO, DPO, and GRPO.

Once those pictures are stable, the equations stop feeling like symbols and start feeling like bookkeeping.

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*, second edition. MIT Press, 2018. http://incompleteideas.net/book/the-book-2nd.html

[2] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[4] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. arXiv:1506.02438, 2015.

[5] John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel. Trust region policy optimization. In *ICML*, 2015.

[6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[7] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *ICML*, 2018.

[8] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[9] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. In *NeurIPS*, 2020.

[10] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-learning. arXiv:2110.06169, 2021.

[11] Lili Chen, Kevin Lu, Aravind Rajeswaran, et al. Decision transformer: Reinforcement learning via sequence modeling. In *NeurIPS*, 2021.

[12] OpenAI. Spinning Up in Deep RL. https://spinningup.openai.com/

[13] Long Ouyang, Jeff Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.

[14] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. arXiv:2305.18290, 2023.

[15] Peiyi Wang, Lei Li, Zhihong Shao, et al. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. arXiv:2312.08935, 2023.

[16] Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.

[17] John Schulman. Approximating KL divergence. Blog post, 2020. http://joschu.net/blog/kl-approx.html